Régression par processus gaussiens pour l'émulation bayésienne de modèles incertains

Hadrien Cardamone Hugo Moncomble Thomas Yilmaz
Alexandre Bayssiere

CentraleSupélec – Processus gaussiens scalables

19 Janvier 2026

Plan

1. Motivation et cadre du problème
2. Scalabilité via Cholesky incomplète
3. Choix des hyperparamètres
4. Optimisation du code
5. Résultats
6. Conclusion et perspectives

Plan

1. Motivation et cadre du problème
2. Scalabilité via Cholesky incomplète
3. Choix des hyperparamètres
4. Optimisation du code
5. Résultats
6. Conclusion et perspectives

Pourquoi des processus gaussiens scalables ?

Motivation : de nombreux modèles du monde réel sont

- complexes, stochastiques, et coûteux à évaluer,
- mais on a besoin de prédictions en beaucoup de points.

Situation typique :

- Un simulateur $\mathcal{M}(x)$ (par ex. modèle physique, Monte Carlo),
- Coût d'évaluation : de la milliseconde à la seconde par x,
- On ne peut pas se permettre de le lancer sur des millions de scénarios.

Idée : construire un **modèle de substitution** :

- Apprendre une approximation rapide $\hat{f}(x)$ à partir d'un nombre limité de simulations.
- L'utiliser pour réaliser des prédicitons plus rapidement et en quantifier l'incertitude.

La régression par processus gaussiens comme substitut

C'est exactement l'objectif de la régression par processus gaussiens (GP) :

- On observe des évaluations bruitées

$$y(x_i) = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

  sur un nombre limité d'entrées x_i.
- On place un **a priori de processus gaussien** sur la fonciton inconnue.
- En conditionnant par les données, on obtient un **GP a posteriori** : pour tout nouveau $x_\star$,

$$y(x_\star) \mid \mathcal{D}_n \sim \mathcal{N}\big(m_n(x_\star),\, \sigma_n^2(x_\star)\big).$$

Ce que l'on gagne :

- Un substitut rapide $\hat{f}(x) = m_n(x)$ approximant le simulateur.
- Une **quantification d'incertitude** intégrée via $\sigma_n^2(x)$.

Objectifs du projet

Question principale :

Comment rendre la régression par processus gaussiens **scalable** tout en préservant ses principaux atouts ?

Objectifs concrets :

- Comprendre le cadre de **régression bayésienne** des GPs.
- Implémenter un substitut GP utilisant une **factorisation de Cholesky incomplète**.
- Quantifier le compromis :
  - précision prédictive vs. coût de calcul,
  - qualité de l'approximation rang faible vs. temps d'exécution.
- Estimer les hyperparamètres du noyau (σ_f, ℓ, σ) via un maximum de vraisemblance (approximatif).

Régression par processus gaussiens : résumé

But : approximer une fonction inconnue $f : \mathcal{X} \to \mathbb{R}$ à partir de données bruitées.

A priori GP :

$$f \sim \mathcal{GP}(m(x), k(x, x')),$$

avec moyenne $m(x)$ (prise à 0 ici) et noyau de covariance $k(x, x')$.

Étant donnés les points $X = (x_1, \ldots, x_n)$ et les valeurs latentes, on a la marginale finie :

$$\mathbf{f}(\mathbf{X}) \sim \mathcal{N}\big(0,\, K(X, X)\big),$$

où $K(X, X)$ est la matrice de Gram avec entrées $K_{ij} = k(x_i, x_j)$.

Noyau utilisé dans ce projet (RBF isotrope) :

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{\ell^2}\right)$$

avec amplitude σ_f et longueur de corrélation ℓ.

Observations, bruit, et distribution prédictive

Nos données sont représentées par :

$$X = \begin{pmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 & \cdots & y_n \end{pmatrix}^\top.$$

On suppose que ce sont des observations bruitées :

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \quad i = 1, \ldots, n.$$

Pour une nouvelle entrée $x_\star$, en utilisant la formule de Bayes :

$$f(x_\star) \mid X, Y \sim \mathcal{N}\big(m_n(x_\star),\, \sigma_n^2(x_\star)\big),$$

avec :

$$m_n(x_\star) = k(x_\star, X)\, K_\sigma^{-1}\, \mathbf{y},$$
$$\sigma_n^2(x_\star) = k(x_\star, x_\star) - k(x_\star, X)\, K_\sigma^{-1}\, k(X, x_\star).$$

où :

$$K_\sigma = K(X, X) + \sigma^2 I_n.$$

Caractéristique clé : le GP fournit **à la fois** une prédiction et une **incertitude**.

Plan

1. Motivation et cadre du problème
2. Scalabilité via Cholesky incomplète
3. Choix des hyperparamètres
4. Optimisation du code
5. Résultats
6. Conclusion et perspectives

GP complet : problème de complexité

Matrice centrale : $K_\sigma = K + \sigma^2 I_n$.

- Stockage : $\mathcal{O}(n^2)$.
- Factorisation de Cholesky : $\mathcal{O}(n^3)$.
- Résolution $K_\sigma \alpha = y$ en $\mathcal{O}(n^2)$ une fois L connu.

Coût typique (sur ma machine) :

- $n = 10\,000$: quelques secondes pour une factorisation / résolution.
- $n = 50\,000$: prendrait des dizaines de minutes et ∼20 Go de RAM rien que pour la matrice de covariance.

Besoin :

- une approximation $K \approx \tilde{K}$,
- de rang effectif faible,
- permettant encore de résoudre $K_\sigma \alpha = y$ efficacement.

Pourquoi une approximation de rang faible de la covariance ?

But : éviter de manipuler la matrice de covariance complète K de taille $n \times n$.

Pour de nombreuses matrices de covariance construites à partir de noyaux lisses :

- les **valeurs propres décroissent** assez vite,
- la plupart de "l'information" (trace) est concentrée dans quelques directions.

Approximation de rang faible :

$$K \approx LL^\top, \qquad L \in \mathbb{R}^{n \times k}, \quad k \ll n.$$

- Chaque colonne de L capture une direction importante de variation.
- Utiliser L au lieu de K réduit le coût algébrique de $\mathcal{O}(n^3)$ à $\mathcal{O}(nk^2)$ (avec Woodbury).

La factorisation de Cholesky incomplète est une manière de construire un tel L **directement à partir de** K.

Cholesky incomplète pivotée : algorithme et critère d'arrêt

Algorithme IC pivoté :

- Initialiser une diagonale résiduelle

$$D(i) = K_{ii}, \quad i = 1, \ldots, n.$$

- Pour $k = 1, 2, \ldots$:
  - choisir le pivot

$$i_k = \arg\max_i D(i)$$

  - calculer la k-ième colonne $L(:, k)$ à partir de K et des colonnes précédentes,
  - mettre à jour la diagonale résiduelle pour tout j non encore sélectionné :

$$D(j) \leftarrow D(j) - L(j, k)^2.$$

Critère d'arrêt basé sur la trace :

$$\mathrm{Tr}(K - LL^\top) \;\leq\; \eta\,\mathrm{Tr}(K) \quad\Longleftrightarrow\quad \sum_{j=1}^{k} \|L(:, j)\|_2^2 \;\geq\; (1 - \eta)\,\mathrm{Tr}(K),$$

avec une tolérance $\eta \in (0, 1)$.

Résolution approchée via l'identité de Woodbury

À partir de IC :

$$K \approx LL^\top, \quad L \in \mathbb{R}^{n \times k}.$$

La covariance bruitée devient :

$$K_\sigma = K + \sigma^2 I_n \approx LL^\top + \sigma^2 I_n.$$

Identité de Woodbury :

$$(LL^\top + \sigma^2 I)^{-1} = \sigma^{-2} \left(I - LM^{-1}L^\top/\sigma^2\right),$$

avec

$$M = I_k + \sigma^{-2} L^\top L \in \mathbb{R}^{k \times k}.$$

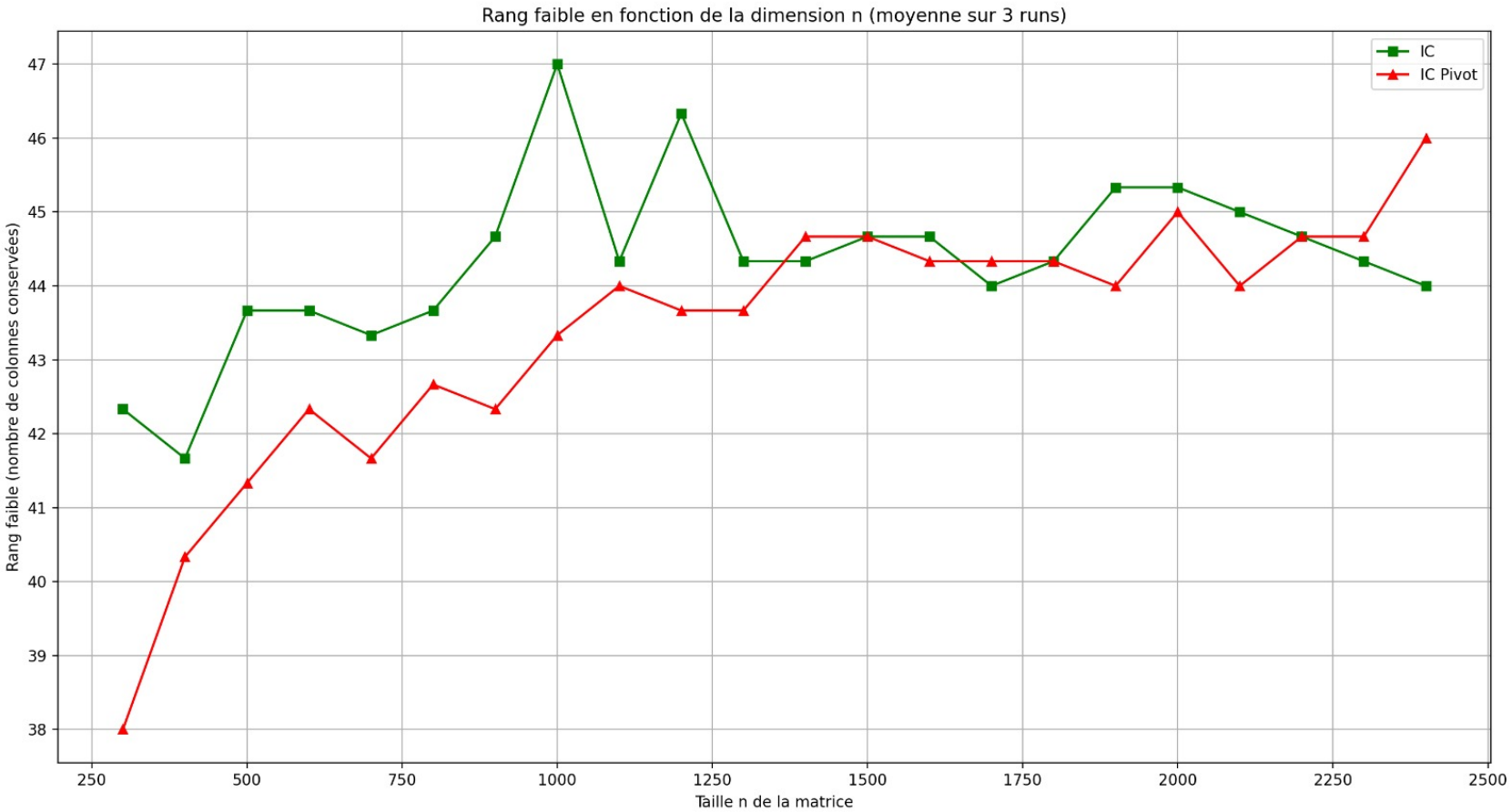
Implémentation dans mon code :

- Calculer L via IC une seule fois.
- Construire $M = I_k + \sigma^{-2} L^\top L$.
- Utiliser Cholesky sur M (taille $k \times k$) pour résoudre

$$\alpha \approx (LL^\top + \sigma^2 I)^{-1} y.$$

Complexité par résolution : $\mathcal{O}(nk^2 + k^3)$.

Un résultat surprenant : Cholesky avec ou sans pivot



Observation : Pour un niveau d'erreur donné, le rang nécessaire est similaire avec ou sans pivot.

Pourquoi ? Le pivot devrait sélectionner les directions les plus informatives, et donc réduire l'erreur plus rapidement, mais ce n'est pas ce qui est observé ici.

Erreur en fonction du rang : avec et sans pivot

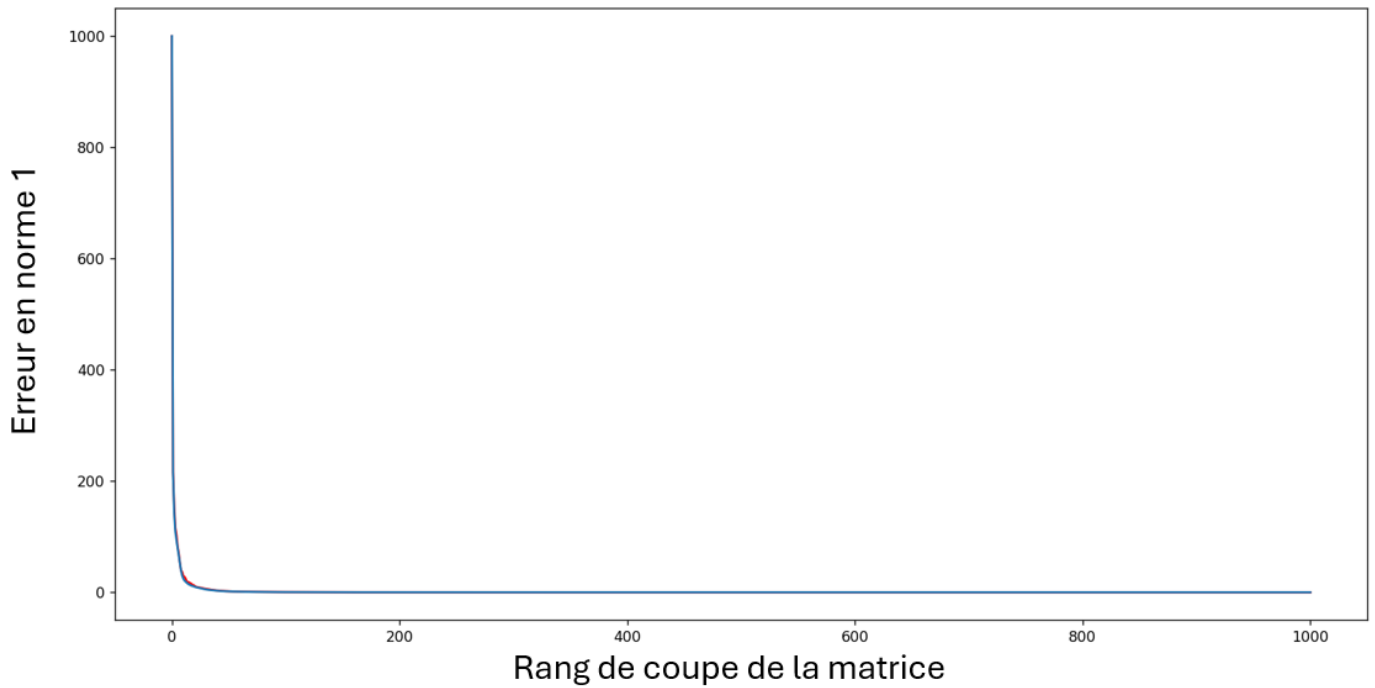
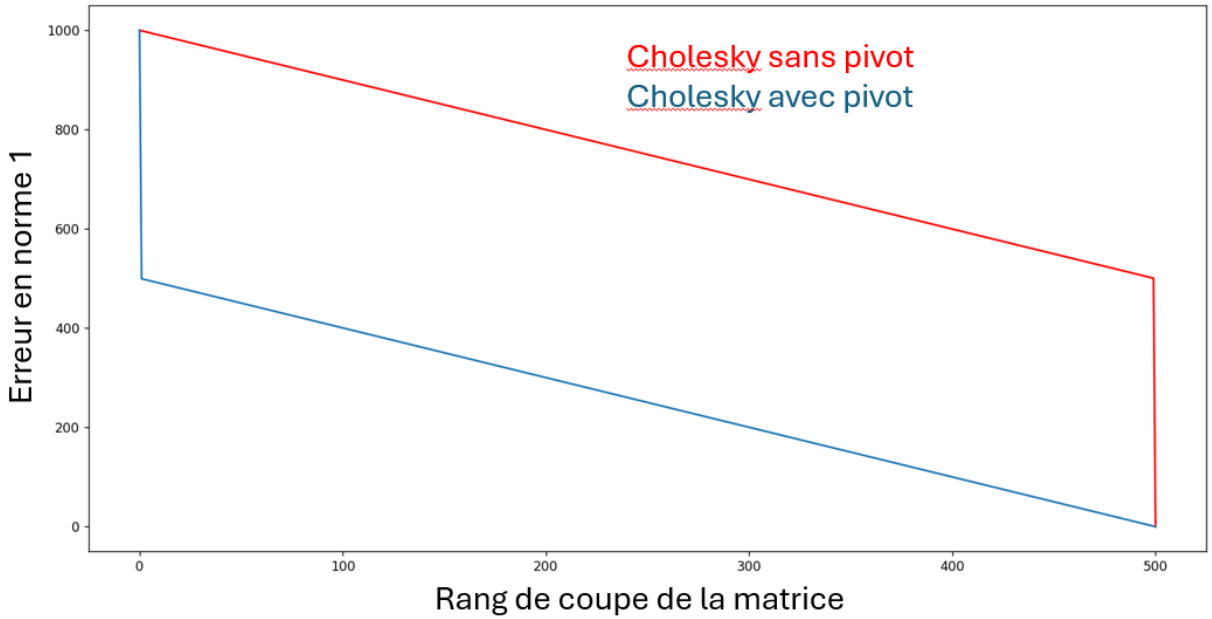


Figure – Erreur de reconstruction selon le rang approximé, pour Cholesky avec pivot (en bleu) et sans pivot (en rouge). Les courbes sont quasiment confondues, quelle que soit la norme utilisée.

Remarque : L'ajout du pivot n'apporte pas d'amélioration significative sur l'erreur, pour toutes les normes testées. Les deux variantes suivent pratiquement la même trajectoire.

Cas particulier : pivot détecte les valeurs dominantes

Configuration : Matrice 500×500 avec valeurs diagonales $= 1$, sauf la dernière diagonale $= 500$. Objectif : vérifier que le pivot de la Cholesky incomplète avec pivot sélectionne la valeur dominante.

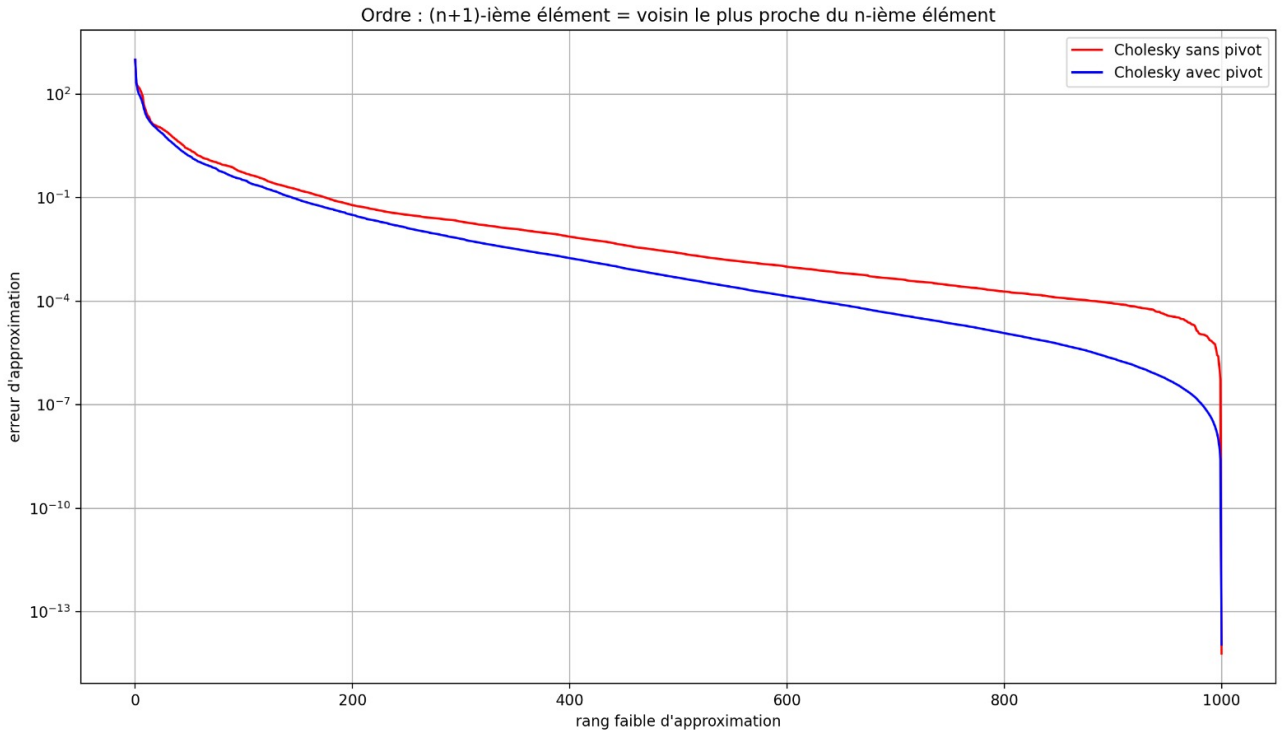


Observation : Le pivot remplit bien son rôle en détectant les éléments les plus "informants" de la matrice.

Cas particulier : colonnes fortement redondantes

Configuration : Matrice de covariance réordonnée pour que chaque colonne soit le plus proche voisin de la précédente.

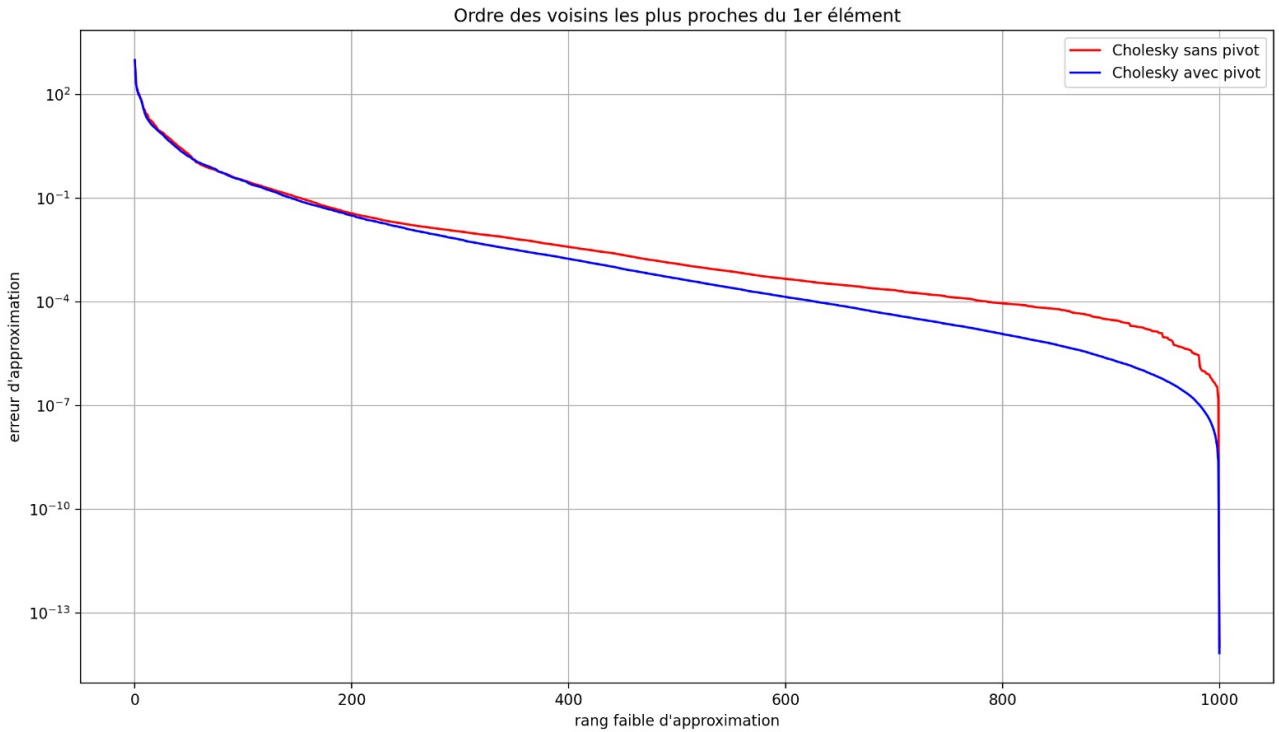
- La méthode **sans pivot** ajoute très peu d'information à chaque étape (colonnes redondantes).
- La Cholesky incomplète **avec pivot** sélectionne rapidement les directions les plus informatives.



Cas particulier : plus proche voisin parmi n points

Configuration : À chaque étape, on sélectionne le plus proche voisin parmi les n points précédents, et non seulement par rapport au dernier point ajouté.

- La Cholesky incomplète **avec pivot** conserve un avantage, mais plus modéré.
- La méthode avec pivot fournit toujours une approximation légèrement meilleure que la version sans pivot.



Plan

1. Motivation et cadre du problème
2. Scalabilité via Cholesky incomplète
3. Choix des hyperparamètres
4. Optimisation du code
5. Résultats
6. Conclusion et perspectives

Noyau RBF et hyperparamètres

Nous utilisons un noyau gaussien (RBF) pour modéliser notre fonction inconnue :

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{\ell^2}\right)$$

Les observations sont bruitées :

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Les hyperparamètres du modèle sont donc :

$$\theta = (\sigma_f,\ \ell,\ \sigma)$$

- σ_f contrôle l'amplitude des variations de f
- ℓ définit la longueur de corrélation, donc la régularité
- σ représente le niveau de bruit dans les observations

But : estimer ces paramètres pour améliorer la précision prédictive du GP.

Estimation empirique de la variance du bruit

Avant d'optimiser la NLL, il est important d'avoir une estimation robuste de σ.

Principe :

- pour des points proches, la fonction f varie peu
- la dispersion locale des observations reflète essentiellement le bruit

Variance locale autour de chaque point x_i^{test} :

$$\widehat{\sigma}_i^2 = \frac{1}{|\mathcal{V}_i|} \sum_{j \in \mathcal{V}_i} \left(y_j - \overline{y}_{\mathcal{V}_i} \right)^2$$

Estimation globale :

$$\widehat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^{N} \widehat{\sigma}_i^2$$

- Cette approche fournit un ordre de grandeur robuste
- Elle peut être utilisée comme valeur fixe ou comme initialisation pour l'optimisation

Vraisemblance marginale du GP

Avec :

$$f \sim \mathcal{GP}(0, k), \quad y \sim \mathcal{N}(0, K_\sigma), \quad K_\sigma = K + \sigma^2 I$$

La log-vraisemblance marginale est :

$$\log p(y|X, \theta) = -\frac{1}{2} y^\top K_\sigma^{-1} y - \frac{1}{2} \log |K_\sigma| - \frac{n}{2} \log(2\pi)$$

Objectif pratique : minimiser la **log-vraisemblance negative** (NLL) :

$$\mathcal{L}(\theta) = \frac{1}{2} y^\top K_\sigma^{-1} y + \frac{1}{2} \log |K_\sigma| + \frac{n}{2} \log(2\pi)$$

Calcul efficace : rang faible, Woodbury et Sylvester

Problème : inversion et déterminant de $K_\sigma \in \mathbb{R}^{n\times n}$ coûteux pour grand n.

Solution : approximation de rang faible et identité de Woodbury :

$$K \approx LL^\top, \quad L \in \mathbb{R}^{n\times k},\ r \ll n$$

$$K_\sigma \approx LL^\top + \sigma^2 I$$

Identité de Woodbury :

$$K_\sigma^{-1} = \sigma^{-2}\Big(I - L(I + \sigma^{-2}L^\top L)^{-1}L^\top\Big)$$

Déterminant (formule de Sylvester) :
On factorise d'abord la petite matrice via Cholesky :

$$I + \sigma^{-2}L^\top L = RR^\top$$

Puis on obtient le log-determinant de K_σ :

$$\log|K_\sigma| = 2n\log\sigma + 2\sum_{i=1}^{k}\log R_{ii}$$

Optimisation des hyperparamètres

Optimisation numérique de la NLL :

`scipy.optimize.minimize`

Méthodes testées :

`L-BFGS-B`, `BFGS`, `CG`, `SLSQP`, `TNC`

Résultat : `L-BFGS-B` fournit la meilleure combinaison de :

- stabilité numérique
- valeur finale de NLL
- temps de convergence

Paramétrisation et transition vers l'optimisation du code

Optimisation sur les log-hyperparamètres :

$$\theta = (\log \ell, \log \sigma_f)$$

Contrainte imposée :

$$\ell \geq 0.8$$

- Évite un noyau quasi diagonal
- Préserve la structure de rang faible
- Stabilise la Cholesky incomplète et accélère le calcul

Constat clé : le coût est dominé par la construction de K et sa décomposition.
$\Rightarrow$ Section suivante : **Optimisation du code pour grandes données**

Plan

1. Motivation et cadre du problème
2. Scalabilité via Cholesky incomplète
3. Choix des hyperparamètres
4. Optimisation du code
5. Résultats
6. Conclusion et perspectives

Optimisation 1 : Woodbury sans inversion explicite

Contexte : après IC, on approxime

$$K \approx LL^\top, \qquad L \in \mathbb{R}^{n\times k}, \quad k \ll n, \qquad K_\sigma \approx LL^\top + \sigma^2 I_n.$$

Identité de Woodbury :

$$(LL^\top + \sigma^2 I)^{-1} = \frac{1}{\sigma^2} I - \frac{1}{\sigma^4} L \left(I_k + \frac{1}{\sigma^2} L^\top L \right)^{-1} L^\top.$$

Point clé : on ne forme jamais l'inverse

Au lieu de construire une matrice dense $(LL^\top + \sigma^2 I)^{-1} \in \mathbb{R}^{n\times n}$, on calcule directement

$$\alpha := (LL^\top + \sigma^2 I)^{-1} y.$$

Calcul pratique :

$$M := I_k + \frac{1}{\sigma^2} L^\top L, \qquad z = M^{-1}(L^\top y), \qquad \alpha = \frac{1}{\sigma^2} y - \frac{1}{\sigma^4} L z.$$

Coût :

$$L^\top y : \ \mathcal{O}(nk), \quad L^\top L : \ \mathcal{O}(nk^2), \quad \text{solve } Mz : \ \mathcal{O}(k^3) \ \Rightarrow \ \mathcal{O}(nk^2 + k^3).$$

Optimisation 2 : éviter la construction de la matrice K

Problème : pour un noyau RBF, construire explicitement la matrice

$$K_{ij} = k_\theta(x_i, x_j) \in \mathbb{R}^{n \times n}$$

coûte :

$$\mathcal{O}(n^2 d) \text{ en calcul.}$$

Pourquoi c'est bloquant ?

L'IC vise un coût proche de $\mathcal{O}(nk^2)$ (avec $k \ll n$). Si on construit K au préalable, le coût quadratique $\mathcal{O}(n^2 d)$ devient dominant et annule une partie du gain.

Objectif : rendre la factorisation IC et l'optimisation des hyperparamètres **scalables** en supprimant la construction explicite de K.

Optimisation 2 : IC *matrix-free*

Observation : l'IC pivotée ne requiert pas toute la matrice K, mais uniquement :

- la diagonale K_{jj},
- les colonnes pivotées $K(:, i_m)$.

Mise à jour IC (colonne m) :

$$L_{j,m} = \frac{1}{L_{i_m,m}} \left(K_{j,i_m} - \sum_{r=1}^{m-1} L_{j,r} L_{i_m,r} \right), \qquad D_j \leftarrow D_j - L_{j,m}^2.$$

Donc : on calcule uniquement la colonne nécessaire

$$K(:, i_m) = \left(k(x_1, x_{i_m}), \ldots, k(x_n, x_{i_m}) \right)^\top$$

sans jamais former K.

Coût global

Si l'IC s'arrête au rang k :

$$\mathcal{O}(knd) \text{ au lieu de } \mathcal{O}(n^2 d),$$

Plan

1. Motivation et cadre du problème
2. Scalabilité via Cholesky incomplète
3. Choix des hyperparamètres
4. Optimisation du code
5. Résultats
6. Conclusion et perspectives

Compromis : rang vs. précision

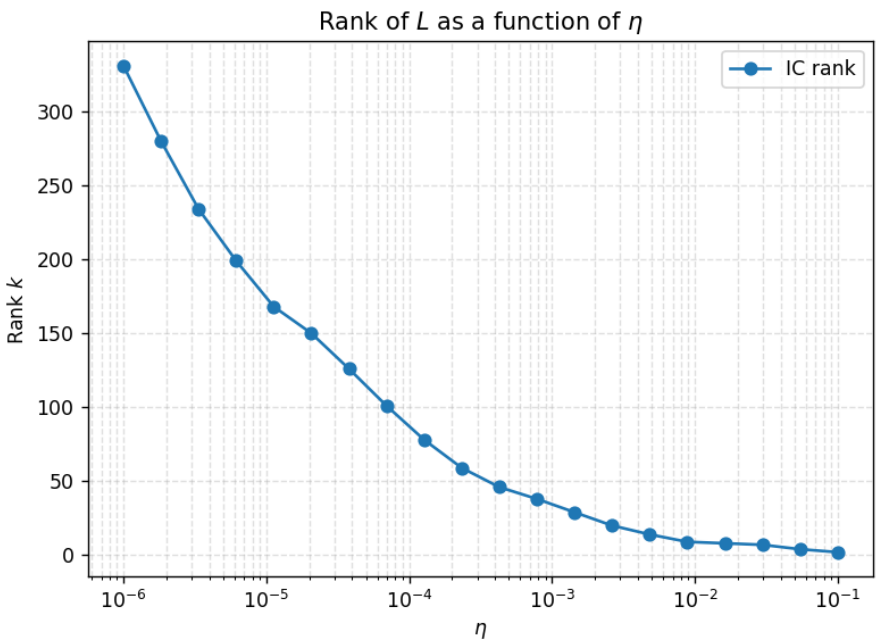


Figure : Rang k vs. tolérance η

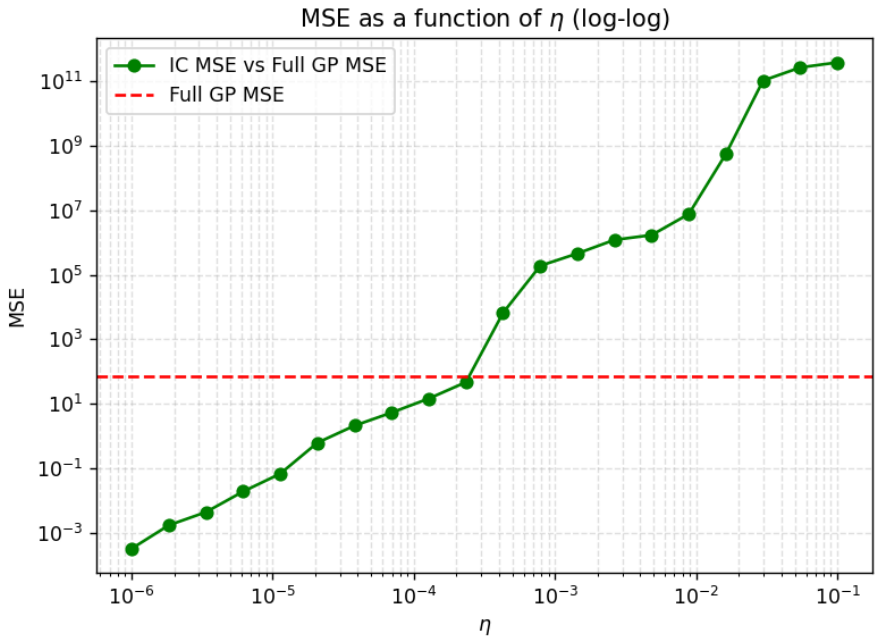


Figure : RMSE vs. tolérance η

Observations (attendues) :

- diminuer η augmente fortement le rang k,
- le RMSE s'améliore et se rapproche du GP complet,
- il existe un **bon compromis** où :
  - $\text{RMSE}_{\text{IC}} \approx \text{RMSE}_{\text{complet}}$,
  - mais $k \ll n$, ce qui donne des accélérations substantielles.

Comparaison des temps : arrêt à eta vs. Cholesky complet

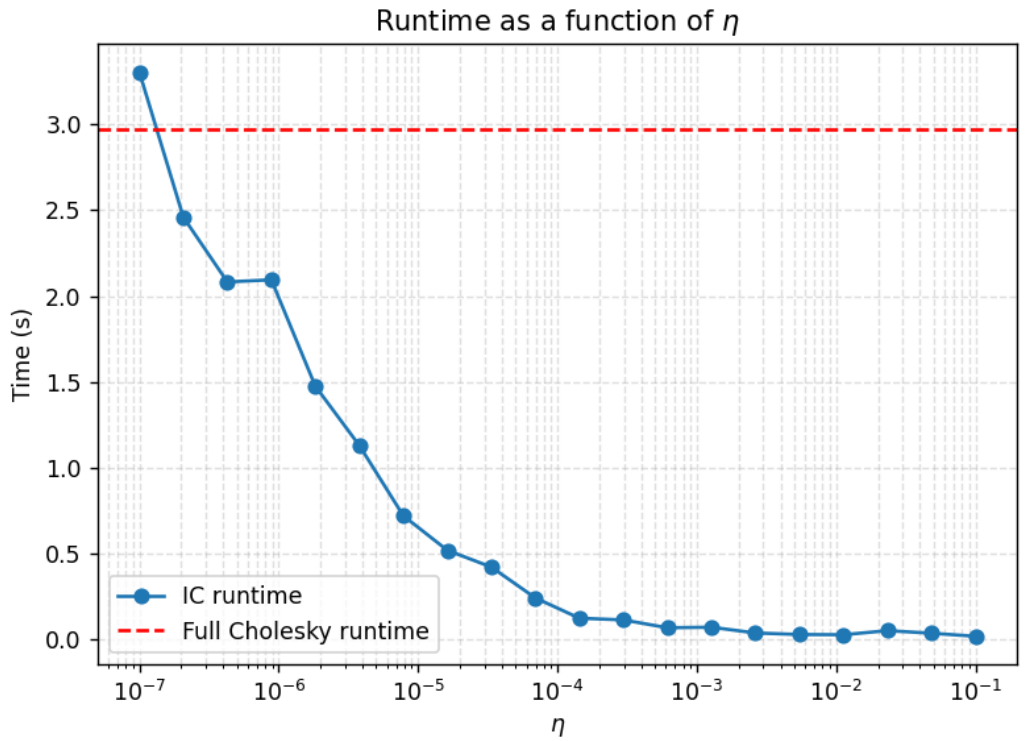


Figure : Temps d'exécution vs. η pour GP complet et GP-IC

Explication de la figure :

- Axe X : η (pourcentage de trace laissé de côté),
- Axe Y : temps d'exécution (secondes).

Visualisation de la NLL (500 points) et localisation du minimum

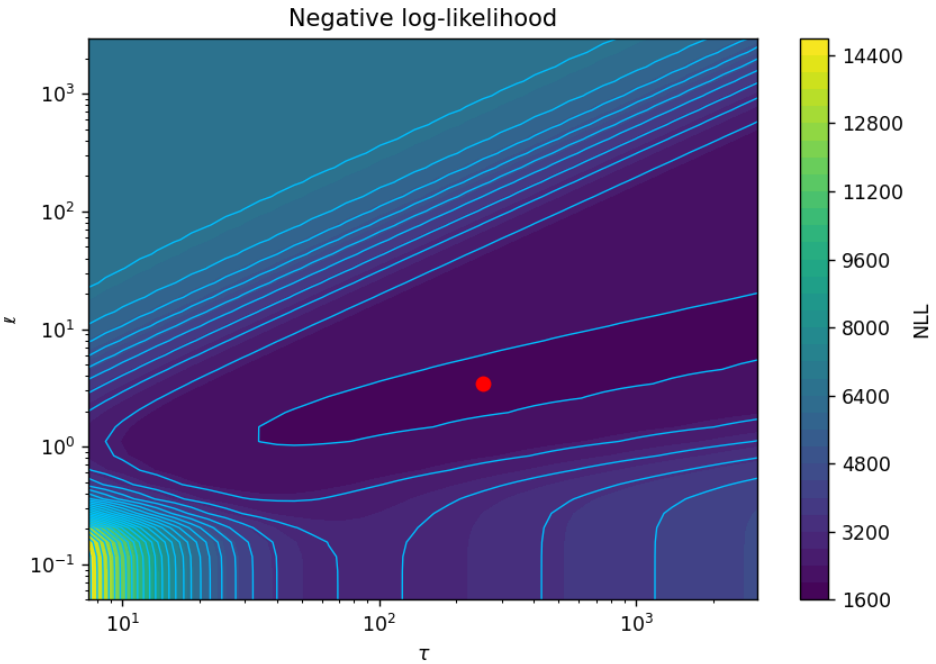


Figure : Carte de niveaux de la NLL en fonction de (τ, ℓ) (échelles log).

Objectif : calibrer les hyperparamètres du noyau RBF

$$k(x, x') = \tau^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right).$$

Lecture de la figure :

- la couleur représente la **NLL** (plus bas = meilleur),
- les courbes sont des **iso-NLL**,
- le **point rouge** indique le **minimum** trouvé sur la grille.

Résultat (minimum) :

$$(\tau^\star, \ell^\star) = (253, 3.5), \qquad \mathrm{NLL}_{\min} = 1600$$

Optimisation : IC sans construire la matrice de covariance

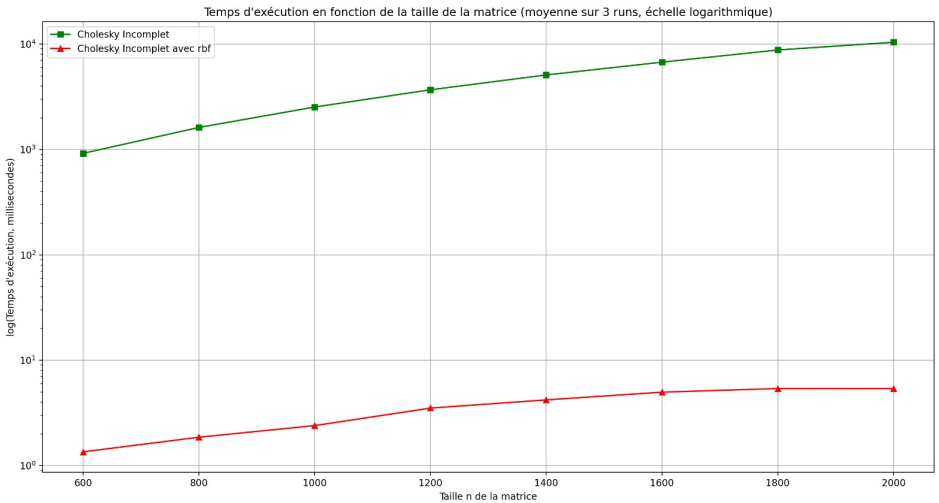


Figure : Temps d'exécution vs taille n (échelle log).

Idée clé : ne jamais former $K(X, X) \in \mathbb{R}^{n \times n}$.

Naïf : construire $K \Rightarrow$ mémoire $\mathcal{O}(n^2)$ + temps élevé.

Optimisé : calculer uniquement les colonnes pivot

$$K(:, i_k) = \big(k(x_1, x_{i_k}), \ldots, k(x_n, x_{i_k})\big),$$

puis mettre à jour L et la diagonale résiduelle D.

Conclusion

Calcul du noyau sans construire la matrice de covariance $\Rightarrow$ **accélération majeure** et meilleure scalabilité.

Prédictions vs. valeurs réelles + incertitude GP

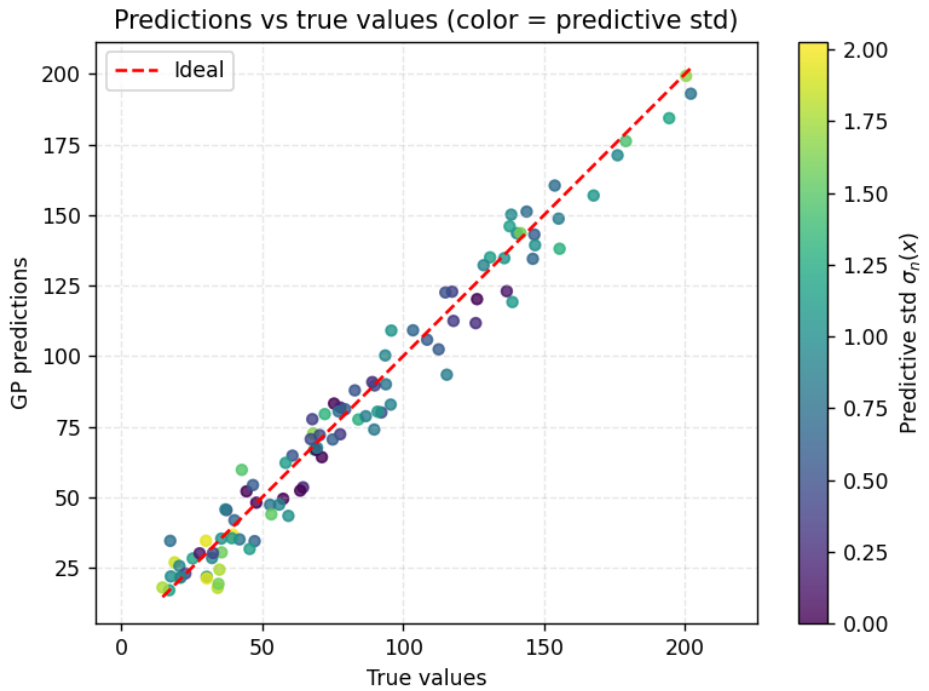


Figure : Vrai vs prédit, coloré en fonction l'écart-type prédictif

Interprétation :

- la plupart des points sont proches de la diagonale (RMSE=8,7),
- les erreurs les plus fortes apparaissent souvent là où le modèle indique une incertitude plus élevée,
- le substitut GP fournit une mesure d'incertitude pertinente.

Dataset bouquetins

- Des points de 3 coordonnées prenant 6 valeurs différentes
- Mille observations par scénario
- Données mélangées
- De nouveaux hyperparamètres

Application du GP aux bouquetins

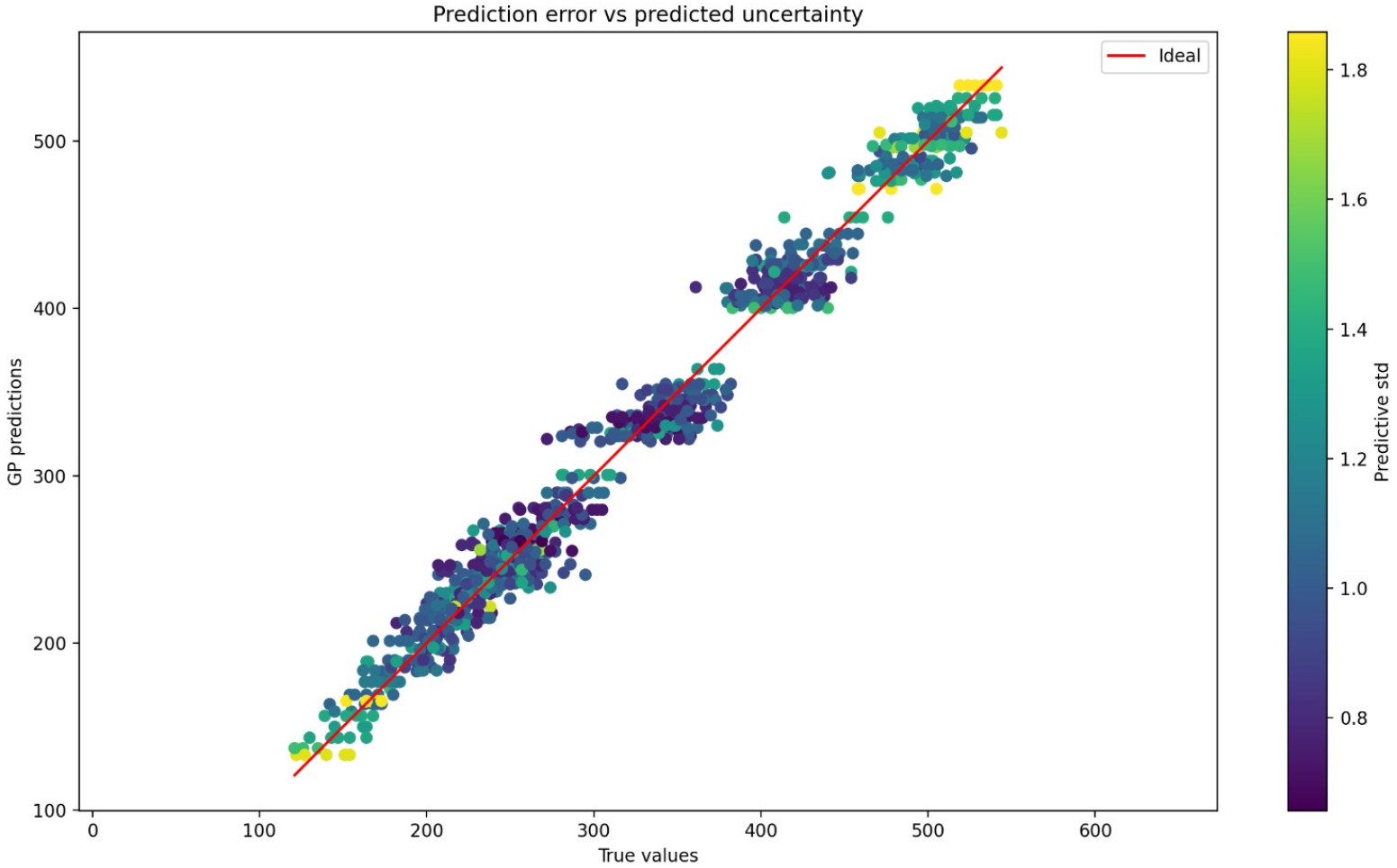


Figure : Vrai vs prédit, coloré en fonction l'écart-type prédictif

RMSE = 14,8478

Plan

1. Motivation et cadre du problème
2. Scalabilité via Cholesky incomplète
3. Choix des hyperparamètres
4. Optimisation du code
5. Résultats
6. Conclusion et perspectives

Conclusion et perspectives

Conclusion :

- La Cholesky incomplète permet une approximation de rang faible efficace de K, rendant la régression GP applicable à des tailles de données plus grandes.
- L'utilisation de Woodbury évite l'inversion explicite et réduit fortement le coût des résolutions et du calcul de la NLL.
- L'approche sans matrice supprime la construction de K et améliore nettement la scalabilité en temps et en mémoire.

Perspectives :

- Tester la méthode sur **davantage de jeux de données** (benchmarks et cas réels) pour valider la robustesse du compromis précision / coût.
- Explorer une **Cholesky incomplète avec side information** (priorité sur certains points, contraintes métier, structure des données) afin de guider le choix des pivots et améliorer l'approximation.
- Comparer avec d'autres approches scalables et étudier leur impact sur l'incertitude prédictive.

Remerciements

Merci pour votre attention

Nous sommes heureux de répondre à vos questions.

Références clés pour ce projet

Algèbre linéaire creuse & approximation de rang faible

- Scott, J. & Tůma, M. (2023). *Algorithms for Sparse Linear Systems*. Birkhäuser.
- Markovsky, I. (2012). *Low Rank Approximation : Algorithms, Implementation, Applications*. Springer.
- Bach, F. & Jordan, M. (2005). "Predictive low-rank decomposition for kernel methods". *ICML*.

Régression, vraisemblance & incertitude

- Wolberg, J. (2006). *Data Analysis Using the Method of Least Squares*. Springer.
- Eggermont, P.P.B. & LaRiccia, V.N. (2009). *Maximum Penalized Likelihood Estimation, Vol. II : Regression*. Springer.

Méthodes à noyaux & processus gaussiens

- Suzuki, J. (2022). *Kernel Methods for Machine Learning with Math and Python*. Springer.
- Liu, H. et al. (2020). "Scalable Gaussian Processes..." (revue sur les GPs à grande échelle).